# Cyber-Attack Report

Florian Selot, Merouane Sennoun, Alwin Amaladasse, Rameshwari Tailor, Keerthanaa Bollipalli, Sokhna Gaye Ndeye, Louis Berges

## I) Introduction

### • Presentation of the project

The goal of this student project is to work on the development of a complete process of ML modelling, from analysing the data to a website development allowing users to provide data for predictions.
The machine learning model should use the dataset provided as the main source for the input and must predict the type of cyberattack. 3 types of attack are identified: DDoS, Malware, Intrusion.

### • Methodology

The report structure will follow the methodology we used to develop the ML model:

- Exploratory Data Analysis (EDA). This part helped us understand the data, the features and what can be a relevant set of features to use as inputs,
- Feature Engineering. Some feature engineering techniques were used such as preprocessing the data to extract granular information from the existing features. It also includes a PCA/MCA analysis to see where the information is and a search of features combinations that would give unbalanced distribution,
- ML Model. From the EDA and the feature engineering we tried several approaches to create the best ML model and compare each model to a baseline.

## II) Exploratory Data Analysis

### • Presentation of the dataset

We have 40000 records of cyber-attacks distributed across 25 variables. The features can be divided into 2 categories:

1. Network-related features. These are features related to the way the attack was conducted: date, attacker's IP address or the browser used to conduct the attack for example. This data is complete (i.e. non missing values).
2. Post-attack features. These features give information on the attack: for instance, what was detected and who was attacked. Given the specificity of each attack, some features may have missing values. For example, an attack did not trigger the alert system and so the feature Alerts/Warning has a missing value. In our case, five features: Malware Indicators, Alerts/Warnings, Proxy Information, Firewall Logs and IDS/IPS Alerts have approximately 50% missing values as reported in Table 1.

| Column Name | Data Type | Non-Null Count | Missing (%) | Description |
|---|---|---|---|---|
| `Malware Indicators` | object | 20000 | **50.0%** | Specific Indicator of Compromise (IoC) was detected. |
| `Alerts/Warnings` | object | 19933 | **50.2%** | Security alert was triggered. |
| `Proxy Information` | object | 20149 | **49.6%** | Details about the proxy used. |
| `Firewall Logs` | object | 20039 | **49.9%** | Firewall logs were generated. |
| `IDS/IPS Alerts` | object | 19950 | **50.1%** | IDS/IPS alert was generated. |

*Table 1 Features with missing values*

The dataset is balanced across the attack type categories. Of the 40000 observations, around 13300 are for each category which represent 33% of the total observations as expected of a well-balanced dataset in which 3 different categories of attack type are identified.

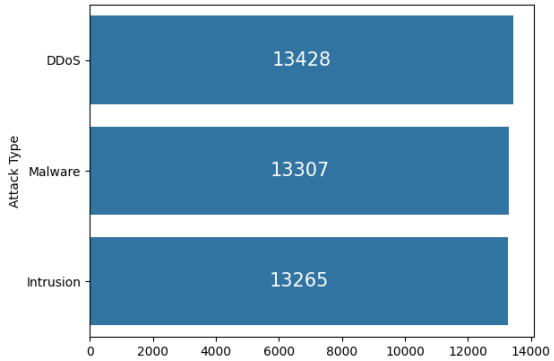



*Figure 1 Records count of the "Attack Type" column*

- ## Numerical and categorical features

We examined the relations and repartition of each other nominal and numerical column relatively to the Attack Type. For the sake of conciseness, we provide a few representative examples in Figure 2. We observe that the distribution remains flat with less than 1% difference between the categories.

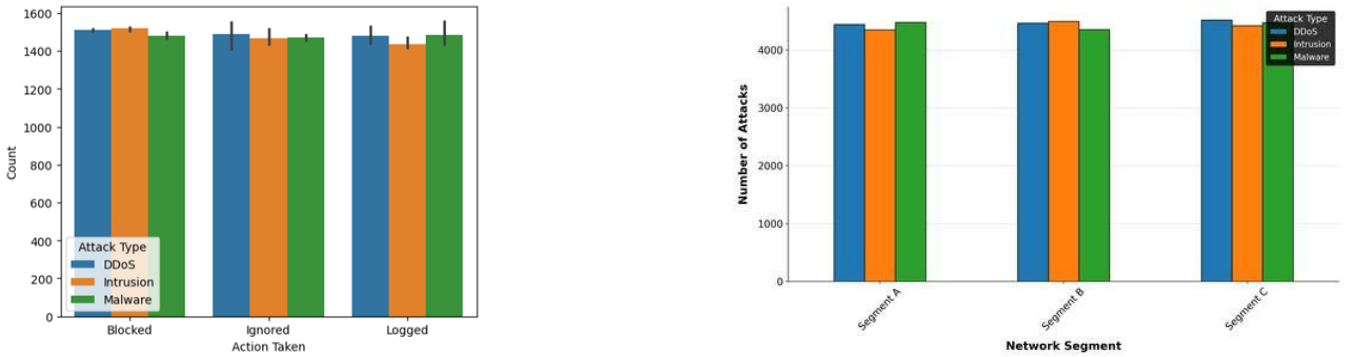



*Figure 2 Records count of the "Action taken" column with error bars based on the severity level (left) and Records count of the "Network Segment" column for each attack type right)*

The same observation applies to all other columns, Source IP Adress, Destination IP Address, Source Port, Destination Port (...) : no specific range of values dominates. This confirms that the dataset is structurally flat, with no strong univariate signal that can be used to distinguish the different attack categories.

- ## Time features

When analysing the time features individually from the Timestamp, no key indicator emerges. The distribution is dense and uniform across all attack types as seen on the figures below. Whether we

group attacks by year, month, day of the week, hour, minute, or second, the proportion of each attack type remains almost identical. No temporal trend, peak, or time-dependent behaviour can be identified.

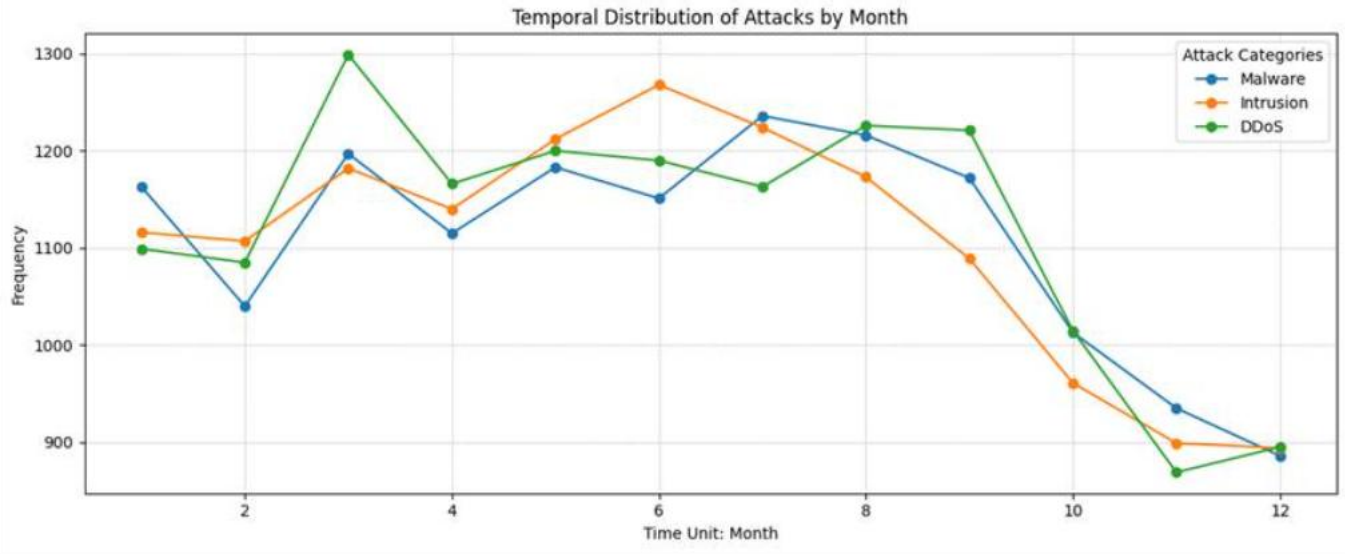



*Figure 3 Records count by month for each attack type*

- ## Device Information

We used the "user_agents" python library on the Device Information column to extract information relative to the victim's environment. It can be divided in 3 categories:

- OS information. It provides information on the OS family (Windows, Linux for example), the version and patch of the OS,
- Browser information. It provides information related to the browser such as the name, version and patch,
- Device information. When possible, it gives information about the device used and the company that fabricates it.

In Figure 4, some of the data we extract from the feature is shown. Looking at the higher level of the information such as the OS family and browser family, we observe the same results as seen previously i.e. there are no differences between the type of attack.

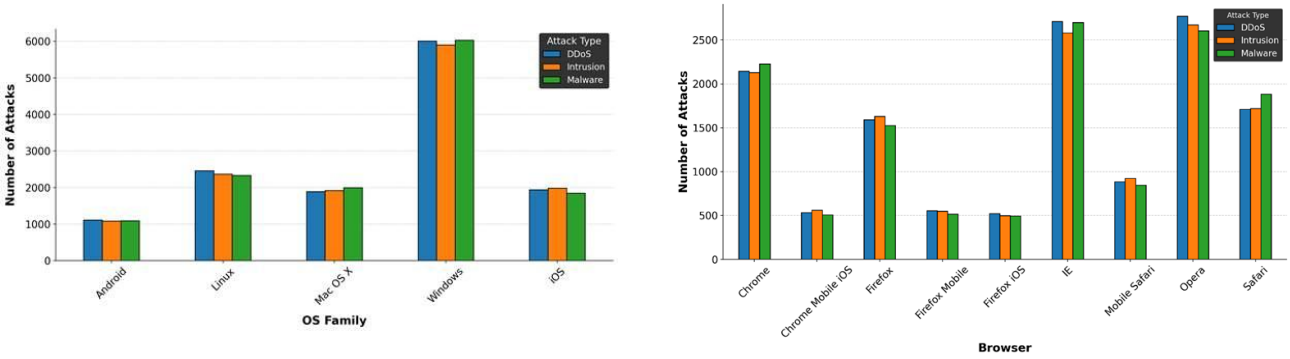



*Figure 4 Attack Type Distribution by OS family (left) and by Browser (right)*

But looking at a lower level such as the windows OS version or the Chrome Version Distribution we can observe differences between the distribution of attack type as shown Figure 5.

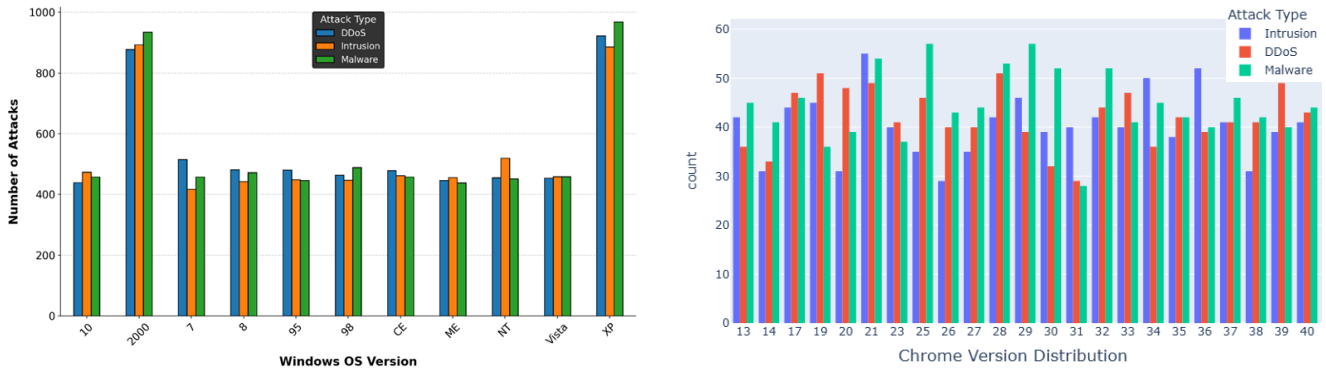



*Figure 5 Attack Type Distribution by Windows OS Version (left) and Chrome Version (right)*

- Geographical Data

Then, after having analysed the distribution of each numerical and nominal column regarding to the target, we focused on looking more specifically on the geographical distribution of the records. Geocoding each address was necessary, as well as dropping the State, because their values were incorrect. We plotted each of the 321 cities in the dataset.

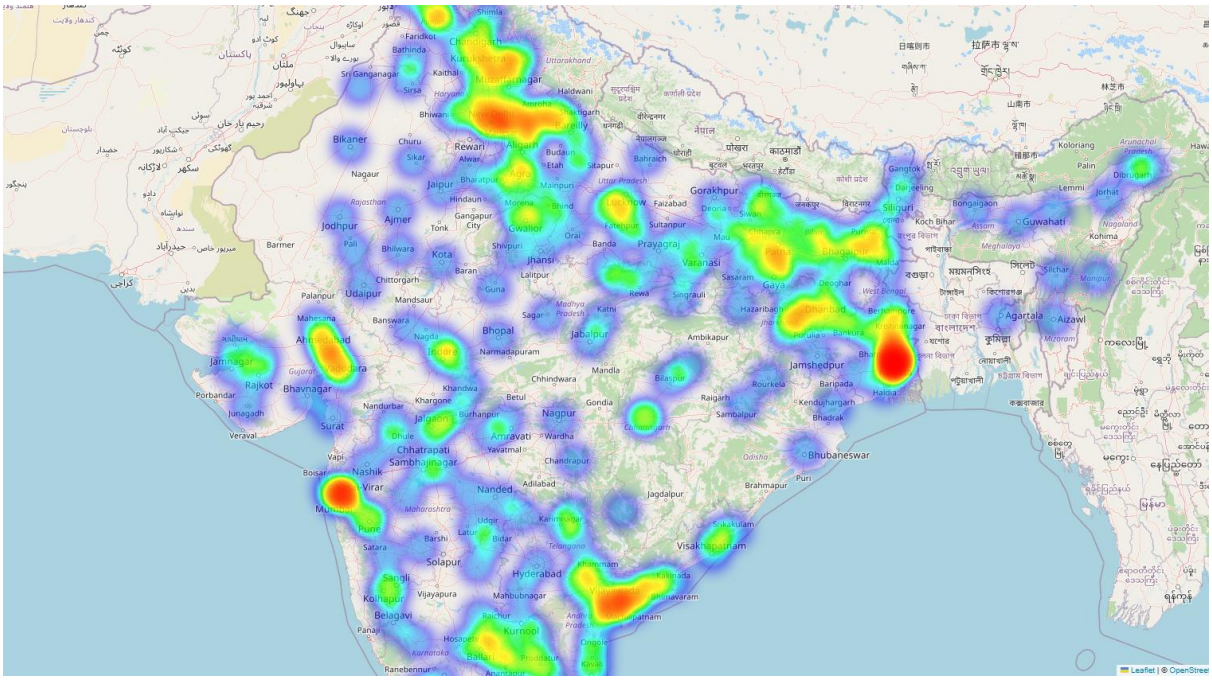


*Figure 6 Geospatial Distribution of Recorded Attacks*

- Conclusion of the EDA & Implications of flatness

From our EDA, we concluded that the features do not seem to be provide information about the type of attack, far from what we expected. The correlation matrix shown Figure 7 resumes it: certain features are interdependent notably IP addresses and ports whereas correlation values for the attack type are close to zero indicating independence from the rest of the dataset. These relationships were expected, as these variables are intrinsically linked to network communication protocols. But we expected to find some relations with the attack type as

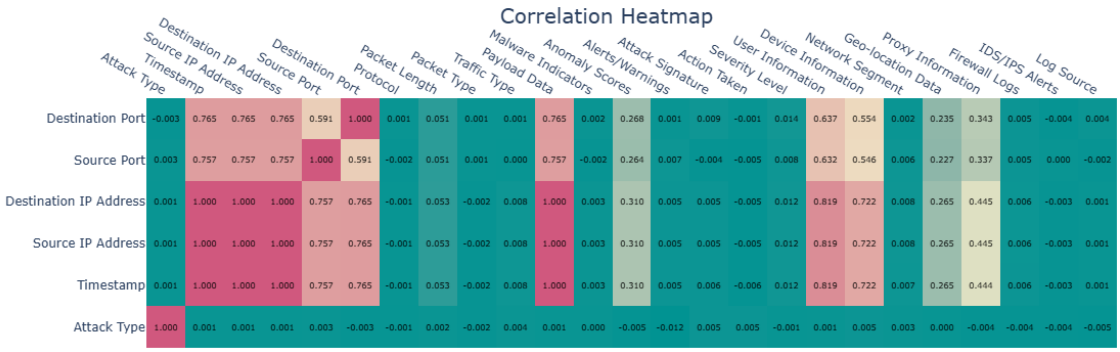



*Figure 7 Part of the Correlation Matrix*

There are several reasons that could explain it:

1. **Balanced Classification**: Real-world cybersecurity data is highly imbalanced (most traffic is benign). A perfectly balanced dataset like this is ideal for training and benchmarking ML models, as it prevents the model from being biased towards the majority class.
2. **Privacy and Anonymity**: Real network data contains sensitive information. Synthetic data allows for realistic-looking features without compromising privacy.
3. **Controlled Environment**: Synthetic data allows creators to control the relationships between variables, ensuring that all attack types are equally represented and that the dataset is "clean" for educational or research purposes.

# III) Feature Engineering

- ## Feature combinations

One of the first technique to create patterns is to combine features: a single feature may not show trends but the combination of several may result in an unbalanced distribution. We tried first to combine features related to the same aspect of the attack. For example, we looked at the combination (Alerts/Warning, IDS/IPS Alerts, Malware Indicators), which are all related to the detection of the cyber-attack. Figure 8 show 5 of the possible tuples of the combination (Traffic Type, Protocol, Packet Type) and the distribution for each. In this case, the distribution within a combination is not balanced but it might not be the case for other combinations.

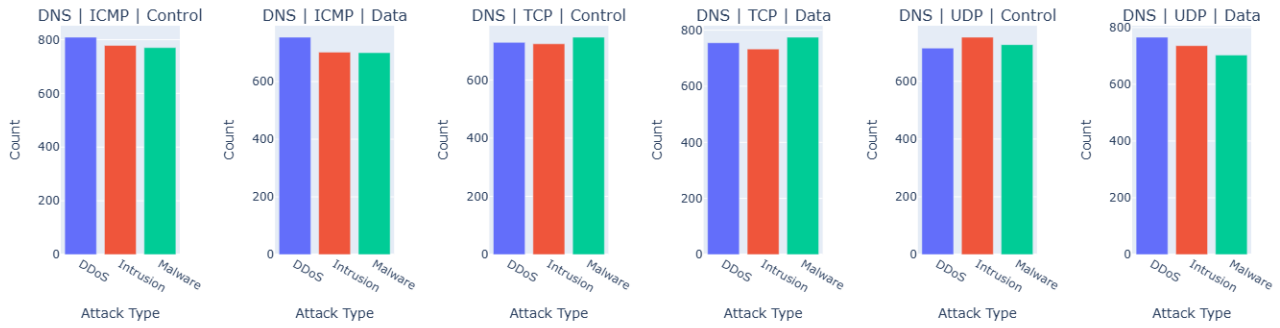



*Figure 8 Attack Type count by the combination of (Traffic Type, Protocol, Packet Type)*

To identify useful feature combinations for handling class imbalance, we automated a filtering process based on how evenly distributed the attack types are across each combination of labels.

Specifically, for each combination, we compared the proportion of observations belonging to the least represented attack type against the most represented one. If the average difference across all label combinations fell between 2% and 99%, we considered that feature combination informative enough to use in feature engineering (lower bound ensures the distribution is uneven enough, while upper bound excludes combinations where one class overwhelmingly dominates).
Applying this filter to the original features only (excluding any derived or atomic features), three three-way combinations emerged as relevant: ('Protocol', 'Network Segment', 'Packet Bin'), ('Packet Type', 'Attack Signature', 'Packet Bin'), and ('Traffic Type', 'Severity Level', 'Packet Bin').

### • Component Analysis

One of the main feature engineering techniques is the Principal Component Analysis or PCA. The objective is to reduce the dimensionality of the input space: some features might be highly correlated and thus can be reduced to a single feature. PCA is commonly well-adapted to numerical features and it equivalent for categorical features is called Multiple Correspondence Analysis (MCA). We used the FAMD function from the Prince library because it handles both numerical and categorical features at the same time.

| component | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| variable | | | | | | | | | | | | | | | | | | |
| num__Source Port | 16% | 0% | 26% | 41% | 9% | 9% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| num__Destination Port | 21% | 18% | 9% | 14% | 16% | 22% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| num__Packet Length | 8% | 52% | 5% | 0% | 12% | 23% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| num__Anomaly Scores | 13% | 18% | 9% | 28% | 23% | 8% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| num__Int Source IP Address | 8% | 12% | 31% | 9% | 40% | 1% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| num__Int Destination IP Address | 34% | 1% | 20% | 8% | 0% | 37% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| cat__Action Taken | 0% | 0% | 0% | 0% | 0% | 0% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% |
| cat__Alerts/Warnings | 0% | 0% | 0% | 0% | 0% | 0% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% | nan% |

*Figure 9 Eigenvalues and features importance obtained by the FAMD algorithm*

The result shown in Figure 9 indicates that the numerical features carry the most information from all features. As this result was unexpected, we conduct the same analysis similar approach designed to handle categorical features called considering categorical features but got the same result with eigenvalues close to zero meaning no importance.

# IV) Modelling & search of best model

### • Baseline Model Configuration

The initial model utilises a Random Forest Classifier integrated into a Scikit-Learn Pipeline. The architecture is designed to handle a heterogeneous dataset through the preprocessing architecture shown Figure 10.

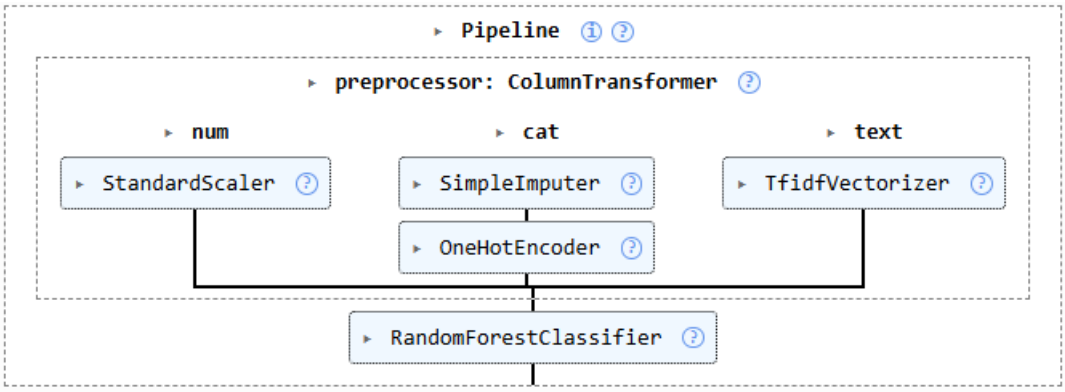



*Figure 10 Baseline model pipeline*

### a. Numerical Encoding

A StandardScaler was used for Source Port, Destination Port, Packet Length, Anomaly Scores, Source IP and Destination IP. It normalises the values and is commonly used to balanced numerical features such as no feature appears to have a higher magnitude.

### b. Categorical Encoding

In our baseline, OneHotEncoder was applied to all string features from low-dimensionality ones like Protocol to fully distinct features like User Data. It creates a new binary column (1 or 0) for every unique category. For example, with the feature “Protocol” where possible labels are TCP,UDP and ICP, if a row is "TCP," the TCP column gets a 1, while UDP and ICMP get 0s. we chose OneHotEncoder over Label or Ordinal encoding because these transformers assign numbers (1, 2, 3...) to categories and mathematical models like Random Forest might interpret these as having a rank or magnitude. Since network protocols and attack signatures have no mathematical order, One-Hot Encoding ensures the model treats each as a distinct, independent entity.

### c. Text Encoding

For the Payload Data, we implemented the TfidfVectorizer function limited to 50 features. TF-IDF stands for Term Frequency-Inverse Document Frequency. It weighs "words" or hex strings based on how often they appear in a packet versus how rare they are across the whole dataset. It is relevant in a cybersecurity context because it puts low weight on common network noise or standard headers that appear everywhere whereas it puts high weight on rare, specific patterns such as an SQL injection fragment or a unique malware signature.

## • Hyperparameter Optimisation: Grid Search

To move beyond default settings, we utilised the model selection part of scikit learn library. It includes various methods to search for the best parameters given an estimator. We used Grid Search CV and HalvingGridSearchCV functions. The former is an exhaustive search that tests every possible combination of parameters (e.g., n_estimators, max_depth, min_samples_split) and uses cross-validation to find the most accurate configuration, while the later search strategy starts evaluating all the candidates with a small amount of resources and iteratively selects the best candidates, using more and more resources.

Using the GridSearchCV on low-dimensional features only give us the following hyper-parameters: max depth is set to None, min_samples_split to 5 and n_estimators to 100. Compared to the default values, only the min_samples_split value changed.
These values are used as default for the other models we developed.

- ## Other Classifiers

There exist other algorithms that can be used to predict the class of such as the Gradient Boosting classifiers. It is a sequence of regression trees that are fit on the negative gradient of loss function, usually a log loss. This type of algorithm retains information especially the errors from the previous steps and try to minimise it at each step.

- ## Alternative Preprocessing Strategies

### *a.* Component Analysis (PCA):

We explored Principal Component Analysis (PCA) to reduce dimensionality. Because One-Hot Encoding creates a "sparse" matrix with many columns, PCA "squashes" these into a smaller set of components that capture the most information, speeding up training, retaining only relevant dimensions and thus reducing noises that would lower the accuracy otherwise.

### *b.* Feature Combination:

We experimented a random forest classifier with some feature combination as preprocessing (e.g., combining Protocol + Traffic Type). This helps the model identify threats that only occur for specific combination, which it might miss if the features remain isolated.

### *c.* Feature Selection (Less Features):

By analysing feature_importances, we removed redundant variables. This creates a *cleaner* model that is less prone to **overfitting** on irrelevant log data.

- Results

All of our tests and researches of better parameters have been unsuccessful. We could not improve the accuracy : it stays close to 33% just like our baseline model as shown in Table 2.

| Model | Description | Accuracy |
|---|---|---|
| Baseline | Random Forest Classifier | 32.84% |
| GridSearchCV | Hyper-parameter research with low dimensionality | 33.47% |
| HistGradBoost | Gradient Boosting Model optimise for large dataset | 32.35% |
| FAMD Model | PCA/MCA transform before Random Forest Classifier | 33.06% |
| Granular Feature | Granular features used | 33.22% |
| Feature Combination | Synthetic feature before Random Forest Classifier | 33.45% |
| Feature Selection | Random Forest => Feature Importance Filter => Random Forest | % |
| Smote Model | XGBboost => SmoteTomek => PCA => Random Forest | 32.34% |

*Table 2 Result of the different models developed during the project*

# V) Learnings from the project

- The SMOTETomek Experiment

One of us found out a model that was giving a 50% accuracy, a huge improvement from the 33%. The preprocessing of the data was the following:

- An XGBoost classifier. Part of the gradient boosting classifier category, it is an optimised version for large dataset from the xgboost library. The idea behind this first step is to reduce the dimensionality of the onehotencoded features.
- A SmoteTomek transformation. SMOTETomek is a hybrid technique that uses SMOTE to create synthetic minority class examples and Tomek Links to clean the boundaries between classes. The objective is to create samples from the minority classes in order to balance the dataset. In fact, we observed that the models were regularly over-predicting the DDoS label which we thought could be counter balanced with this solution.
- A PCA analysis. To reduce the dimension after to reduce time and optimise the model over the newly directions.

This preprocessing is designed to improve performance on unbalanced dataset where one category is over-represented compared to the other categories and we thought it was the reason it worked because our models tend to over-predict the DDoS category (i.e. we observe better precision and recall for the category)
However, we discovered a critical error on our process: we had applied SMOTETomek to the entire dataset before splitting it into training and testing. Information from the test set "leaked" into the training set through the synthetic data points. The model wasn't actually getting better; it was just recognizing data it had already "seen.

- The rule-based method

We initially implemented a rule-based strategy to classify the traffic by leveraging the existing features in the dataset to construct a deterministic signal. In other words, we attempted to combine the available indicators and metadata to manually derive a new categorical label reflecting the presumed attack type. This engineered signal was intended to capture patterns in the data through predefined logic rather than through a learned model.

However, this rule-based strategy did not produce meaningful or generalisable results when evaluated against the original target column. The only way to obtain acceptable performance was either to explicitly incorporate the original attack type column into the rule construction process, or to redefine the prediction target by generating a new column (new attack type column) directly from these deterministic rules and then training the model to predict this engineered label. In both cases, the model was effectively learning patterns that were already embedded in the labelling logic, resulting in circular reasoning and data leakage. Because these approaches artificially inflate performance and do not reflect true predictive capability, they were considered methodological shortcuts and were therefore discarded.

# VI) Conclusion

This project aimed to develop a machine learning model capable of detecting cyberattack types. We developed a structured pipeline including exploratory data analysis, feature engineering and hyperparameter optimisation through extensive research. Despite these efforts, our best resulting model failed to perform better than a random baseline.

Our best explanation is that the dataset does not provide discriminative features even after using different feature engineering methods to create a signal. In this case hyperparameter tuning cannot compensate for the lack of feature informativeness.

While the outcome is disappointing, we did have the opportunity to learn the common mistakes that we may encounter when developing ML models such a data leakage.